# HDOC Day-63

## 1. Problem Understanding

Write a C program that reads the size and elements of a variable-length integer array, nums[]. The array contains distinct integers from 0 to size, with exactly one number missing. Pass the array to a function named missing_number() using its address. Use pointers to pass the array between the calling function and missing_number(). The function missing_number() must find and display the missing integer from the range 0 to size. Assume all elements are distinct and exactly one number is missing. Display the result as an integer.

Input: The size n, followed by n distinct integers from the range 0 to n.

Output: The one missing integer from the range 0 to n.

Approach: Compute the expected sum of integers from 0 to n and subtract the actual sum of the array. The difference is the missing number.

## 2. Standard Test Cases

| Test Case | Size (n) | Array Input | Expected Output |
|---|---|---|---|
| 1 | 3 | 3 0 1 | 2 |
| 2 | 5 | 0 1 2 3 4 | 5 |
| 3 | 4 | 1 2 3 4 | 0 |
| 4 | 6 | 0 1 2 4 5 6 | 3 |
| 5 | 1 | 0 | 1 |

## 3. Algorithm

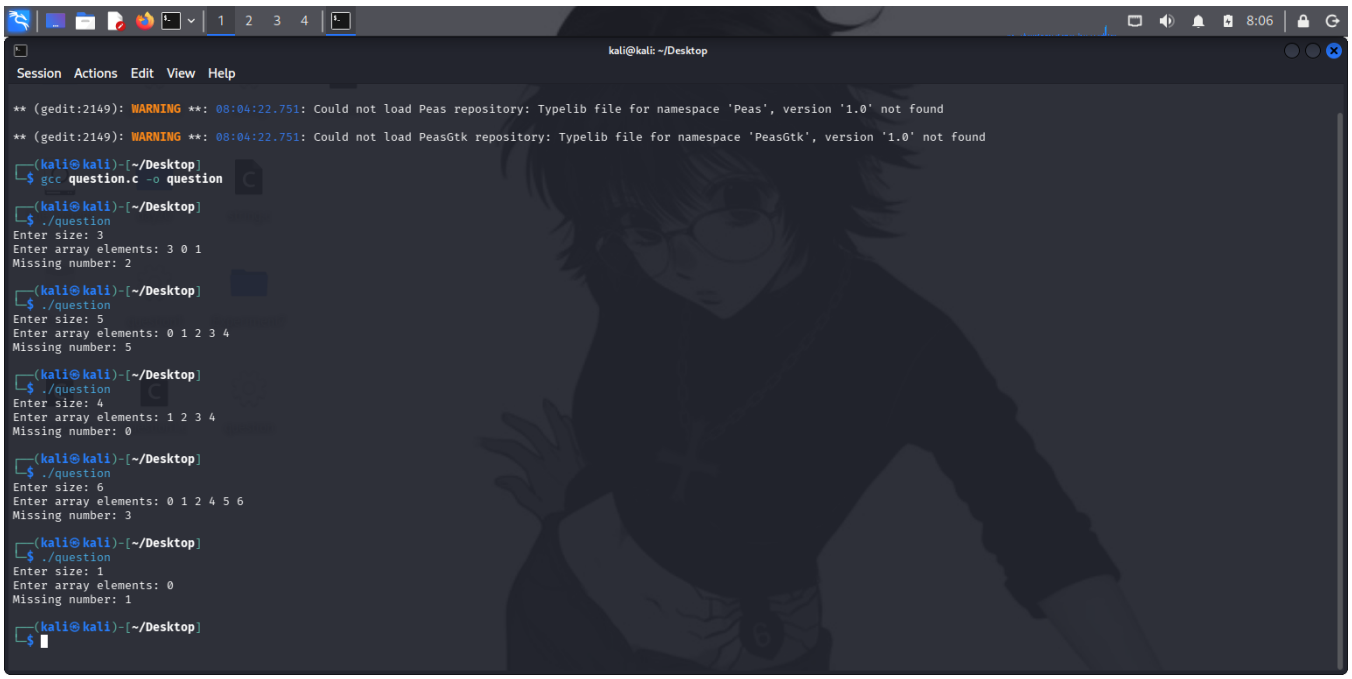
1. Read the size n of the array.
2. Declare a variable-length integer array nums[n] and read its n elements.
3. Pass the address of nums to missing_number() along with n.
4. Inside missing_number(), calculate expected_sum = n * (n + 1) / 2.
5. Using the pointer, add all elements of nums[] to obtain actual_sum.
6. Compute missing = expected_sum - actual_sum.
7. Return the missing number and display it in main().

## 4. C Program

```
#include <stdio.h>
int missing_number(int *nums, int n)
{
   int expected_sum, actual_sum = 0;
   int i;
   expected_sum = n * (n + 1) / 2;
```

```
    for(i = 0; i < n; i++)
    {
        actual_sum = actual_sum + *(nums + i);
    }
    return expected_sum - actual_sum;
}
int main()
{
    int n, i, result;
    printf("Enter size: ");
    scanf("%d", &n);
    int nums[n];
    printf("Enter array elements: ");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &nums[i]);
    }
    result = missing_number(nums, n);
    printf("Missing number: %d\n", result);
    return 0;
}
```

## 5. Compilation and Execution Evidence



## 6. Verification of Test Cases

| Test Case | Size (n) | Array Input | Expected | Actual | Result |
|---|---|---|---|---|---|
| 1 | 3 | 3 0 1 | 2 | 2 | Pass |
| 2 | 5 | 0 1 2 3 4 | 5 | 5 | Pass |
| 3 | 4 | 1 2 3 4 | 0 | 0 | Pass |
| 4 | 6 | 0 1 2 4 5 6 | 3 | 3 | Pass |
| 5 | 1 | 0 | 1 | 1 | Pass |